



EMHIP — Ethnicity & Mental Health Improvement Project

EMHIP Technical Handover

Version 1.0

30 September 2026

For: developers and IT staff taking over EMHIP

Contents

1. Purpose and scope
2. Architecture overview
3. Solution layout
4. Key design decisions
5. Data model overview
6. API overview
7. Front-end structure
8. Authentication and security
9. Background workers
10. Configuration reference
11. Local development
12. Database and migrations
13. Testing
14. Working practices
15. Known technical debt and recommendations

This document explains how EMHIP is built so that a new development team can change it safely. It covers the architecture, the code layout, the main design decisions, the data model, the API and front end, security, background workers, configuration, local development, migrations, testing and the known technical debt. Running the live server (deploying, backups, restores, troubleshooting) is covered in document 06, the Deployment and Operations Runbook.

1. Purpose and scope

EMHIP is a case-management web application for community mental-health hubs. Staff register the people they support (called **guests**), record contacts and casework notes, manage clinical risk and urgent cases, run a multidisciplinary team (MDT) queue, and report on reach and outcomes. The staff roles are Community Mental Health Worker (CMHW), Community Psychiatric Nurse (CPN), Hub Manager and Admin.

This document describes the code in the `azuree123/emhip_new` repository on 30 September 2026, including the changes from the customer's round 1 testing feedback: dashboard and report drill-throughs to the guest list (`GET /guests?segment=`), plain-English audit descriptions, the report cohort filter, the new Excel sheets, the stale-bundle reload handler and the hidden Documents menu item. Every fact was checked against the code on that date.

Related documents in the pack:

- **04 Administrator Guide:** how to use the Settings, Hub Workers and Roles screens described here.
- **06 Deployment and Operations Runbook:** the server, deployments, backups and restores.
- **08 Known Issues and Recommendations:** the user-facing view of the issues in section 15.
- **09 UK GDPR Compliance Register** (`docs/uk-gdpr-compliance.md`): how data-protection requirements map to the code.

1.1 Terms used in the code and on screen

Some screen labels were changed at the customer's request without renaming the code. Use the code name when searching the repository.

Screen label	Code name	Notes
Guest	<code>Guest</code>	A person supported by a hub
New, Active, Inactive	<code>GuestStatus.New</code> , <code>Active</code> , <code>OnHold</code>	"Inactive" is the display name of <code>OnHold</code>
Urgent	<code>Guest.IsUrgent</code>	A flag, not a status
Contact, scheduled contact	<code>FollowUp</code>	"Contact" replaced "Follow-up" on screen
Contact (recorded)	<code>Contact</code> , <code>CaseworkNote</code>	A logged interaction; casework notes use SBAR
Pathway	<code>GuestPathway</code>	Mental Wellbeing, Clinical Support, Community Recovery
Referral category	<code>PathwayCategory</code>	Practical-support referrals (housing, employment, ...). Not pathways: hidden from every screen on 30 Sep 2026 (the only pathways are the three <code>GuestPathway</code> values); data and <code>POST /guests/{id}/pathway</code> referral API kept
DIALOG scores	<code>DialogAssessment</code>	11 life domains scored 1 to 7
Guest reference G-1001	<code>Guest.GuestNumber</code>	From the SQL sequence <code>GuestNumbers</code>

Screen label	Code name	Notes
Hub	Hub , HubId	Every guest and staff member belongs to one hub

2. Architecture overview

EMHIP runs as four Docker containers behind a host nginx that terminates TLS.

Component	Technology	What it does
Host nginx	nginx on Ubuntu 24.04	TLS on ports 80 and 443 (Let's Encrypt); proxies to <code>127.0.0.1:8080</code>
<code>client</code> container	nginx 1.27 (Alpine) serving the Angular 22 build	Serves the single-page app, proxies <code>/api/</code> and <code>/hubs/</code> to the API, sets browser security headers
<code>api</code> container	ASP.NET Core on .NET 10	REST API, SignalR hub, sign-in (Identity and JWT), database migrations at startup
<code>workers</code> container	.NET 10 generic host	Five background services: outbox relay, escalation, follow-up scheduler, report materialiser, engagement status
<code>sqlserver</code> container	SQL Server 2022 (Express edition in production)	The <code>Emhip</code> database, stored in the <code>sqlserver-data</code> Docker volume
Email provider	SMTP (MailKit), Amazon SES v2 or Mailgun	Transactional email; chosen on the Settings page
Document storage	Local disk, Amazon S3, S3-compatible, Azure Blob or Google Cloud Storage	Uploaded files; chosen on the Settings page

2.1 How a request flows

1. The browser loads the app from `https://emhip.brainshub.co.uk`. The host nginx passes the request to the `client` container.
2. The `client` nginx serves `index.html` and the hashed JavaScript and CSS bundles. `index.html` is never cached; bundles are cached for 30 days.
3. The app calls the API on the same origin under `/api/...` with a JWT in the `Authorization` header. The `client` nginx strips `/api` and forwards to `http://api:8080/`, so the API's own routes have no `/api` prefix (for example `/guests`, `/health`).
4. The API authenticates the token, checks the permission policy on the controller action, and sends a MediatR command or query.
5. Commands write through EF Core. Queries read through read services (hand-written Dapper SQL for the two biggest lists, EF Core no-tracking queries elsewhere).
6. Every successful GET under `/guests/{id}` or `/urgent-cases/{id}` is written to the `AuditEvents` table by `AuditReadLoggingMiddleware`; every write is audited by `AuditSaveChangesInterceptor`.

Because the browser only ever talks to one origin, CORS is only relevant for local development, where the Angular dev server on port 4200 calls the API on port 5299.

2.2 Live urgent-case updates (SignalR)

Only the API holds browser connections, but escalations are processed in the workers. The path is:

1. A risk assessment with any flag set raises `RiskFlagRaisedEvent` (`RiskAssessment.cs`). Resolving an urgent case raises `UrgentCaseResolvedEvent` (`Guest.cs`).
2. `OutboxSaveChangesInterceptor` writes each event to `OutboxMessages` in the same transaction as the change.
3. `OutboxRelayWorker` polls every 2 seconds and publishes new rows onto an in-process channel.
4. `EscalationWorker` updates the `UrgentCases_ReadModel` table, then calls `POST http://api:8080/internal/urgent-cases/notify` (or `notify-resolved`) with the `X-Internal-Secret` header.
5. `InternalNotificationsController` checks the secret in constant time and broadcasts `urgentCaseEscalated` or `urgentCaseResolved` to the SignalR group `hub-{hubId}`.
6. The browser's `UrgentCasesHubService` receives the event; the Urgent Cases screen and the menu badge refresh.

The hub endpoint is `/hubs/urgent-cases`. Browsers cannot set headers on a WebSocket handshake, so the token is sent as `?access_token=` and read by the JWT handler for paths under `/hubs`. A user joins only the group for the hub in their own token.

2.3 Email

`IService` (`Emhip.Infrastructure/Email/EmailService.cs`) renders a stored template and hands it to the active provider built by `EmailProviderFactory` from the `email.*` settings. When the provider is `None` (the default), messages are logged and skipped. Send failures are logged and returned, never thrown, so an email problem never fails the clinical action that triggered it.

Template key	Sent by	When
<code>password-reset</code>	API, <code>AuthController</code>	Forgot password
<code>account-created</code>	API, <code>AdminUsersController</code>	An admin creates a staff account
<code>test-email</code>	API, <code>SettingsController</code>	Send test email on the Settings page
<code>urgent-case-raised</code>	Workers, <code>EscalationWorker</code>	A guest is escalated, if <code>email.notifyOnUrgentCase</code> is on
<code>follow-up-overdue</code>	Workers, <code>FollowUpSchedulerWorker</code>	Contacts newly become overdue, if <code>email.notifyOnOverdueFollowUps</code> is on

Templates live in the `EmailTemplates` table, seeded by `EmailTemplateSeeder` (it only inserts missing keys, so admin edits survive deploys).

Note: `Program.cs` also registers `ISender` with `LoggingEmailSender` (`Emhip.Api/Auth/ISender.cs`). Nothing uses it; it is left over from before `IService` existed and can be deleted. The repository README still describes it as the reset-email path, which is out of date.

2.4 Document storage

`IDocumentStorageFactory` (`Emhip.Infrastructure/Storage/DocumentStorageFactory.cs`) builds the storage client from the `documents.storage.*` settings: `LocalDocumentStorage`, `S3DocumentStorage` (Amazon S3 and S3-compatible services such as Contabo or MinIO), `AzureBlobDocumentStorage` or

`GcsDocumentStorage` . Built clients are cached in the singleton `DocumentStorageClientCache` , keyed on the settings, so a settings change produces a new client.

Every `DocumentVersion` row records its own `StorageProvider` , `StorageKey` and SHA-256 hash. Downloads use the version's provider, not the current setting, so switching provider only affects new uploads; old files are still read with the saved settings for their provider. Do not clear an old provider's credentials while files remain there.

Upload size is limited in three places: the Settings value `documents.upload.maxFileSizeMb` (default 25), the API's `[RequestSizeLimit(524_288_000)]` on the upload endpoints, and `client_max_body_size 512m` in `client/nginx.conf` .

3. Solution layout

The solution file is `Emhip.slnx` . Project references run one way: Domain, then Application, then Infrastructure, then Api and Workers.

Path	What lives there
<code>src/Emhip.Domain</code>	Entities, enums, domain events, the permission catalogue (<code>Authorization/Permissions.cs</code>) and built-in role names. No dependencies
<code>src/Emhip.Application</code>	MediatR commands and queries by feature folder, DTOs, FluentValidation validators, abstractions (<code>IAppDbContext</code> , <code>ICurrentUser</code> , <code>IEmailService</code> , <code>IDocumentStorage</code>), <code>SettingsCatalog</code> , <code>GuestSegments</code> , <code>AuditDescriptions</code> , <code>ReportCohortFilter</code>
<code>src/Emhip.Infrastructure</code>	<code>EmhipDbContext</code> , entity configurations, migrations, interceptors, seeders, read services (<code>Reads/</code>), read-model entities, email providers, storage providers, settings service, Excel builder
<code>src/Emhip.Api</code>	Controllers, Identity and JWT setup, permission handler, SignalR hub, middleware, <code>Program.cs</code> , Dockerfile
<code>src/Emhip.Workers</code>	The five <code>BackgroundService</code> classes, the in-process outbox channel, the HTTP notifier that calls the API, Dockerfile
<code>tools/Emhip.Seeder</code>	Console app that bulk-loads synthetic data with <code>SqlBulkCopy</code> and Bogus. Never run it against production
<code>tests/Emhip.UnitTests</code>	xUnit tests for the domain and application layers
<code>tests/Emhip.IntegrationTests</code>	One <code>WebApplicationFactory</code> test of <code>/health</code>
<code>client</code>	Angular 22 app, its Dockerfile and <code>nginx.conf</code>
<code>deploy</code>	<code>deploy.sh</code> , <code>backup.sh</code> , systemd units and the server README
<code>docker-compose.yml</code> , <code>docker-compose.prod.yml</code>	The stack; production layers the second file on the first
<code>.env.example</code>	Names of the secrets Compose needs; the real <code>.env</code> is never committed
<code>docs</code>	This handover pack, the UK GDPR register and the round 1 feedback sheets

Path	What lives there
project	Design handoff, architecture brief, screen designs, flow specification and the functional requirements (v2) PDF

Application feature folders are: `Audit`, `Common`, `Contacts`, `CustomFields`, `Dashboards`, `Documents`, `Emails`, `FollowUps`, `Guests` (with sub-folders for commands, queries and DTOs, and for actions, care plans, caseload, casework, clinical, compliance, CPN, DIALOG and pathways), `Lookups`, `Mdt`, `Migration`, `Notes`, `Reports`, `Settings` and `UrgentCases`.

4. Key design decisions

4.1 CQRS split

Writes and reads take different paths.

- **Commands** are MediatR handlers that load aggregates through `IAppDbContext` (EF Core, change tracking) and call domain methods. `ValidationBehavior` runs FluentValidation validators first; a `ValidationException` becomes a 400 ProblemDetails response through `ValidationExceptionHandler`. `DomainExceptionHandler` maps `KeyNotFoundException` and `FileNotFoundException` to 404 and `InvalidOperationException` to 400.
- **Queries** go to read services in `Emhip.Infrastructure/Reads`. The guest list (`GuestReadService.GetGuestListAsync`) and the follow-up queue (`FollowUpReadService`) use hand-written Dapper SQL. All other reads, including reports and dashboards, use EF Core `AsNoTracking()` LINQ queries.
- **Controllers** are thin: they read the caller's hub from `ICurrentUser`, send one request and shape the HTTP response.

4.2 Keyset pagination

Long lists never use OFFSET. `KeysetPage<T>` carries `Items`, `NextCursor`, `HasMore` and `TotalCount` (only on the first page). `KeysetCursor` encodes the last row's sort key as base64url JSON, which clients treat as opaque. The guest list sorts by `(LastName, FirstName, Id)` and is backed by the index `IX_Guests_Keyset` on `(HubId, LastName, FirstName, Id)`; the page size is clamped to 1 to 200 (default 50). Keyset paging is also used for the follow-up queue, hub-wide contact history, a guest's contact history and the document register.

4.3 Guest-list segments (drill-throughs)

Dashboard and report counts link to `/guests?segment=<key>`, optionally with `clinicalPathway=` and the demographic filters. The keys are defined in `Emhip.Application/Guests/GuestSegments.cs` and mirrored with display labels in `client/src/app/core/guest-segments.ts`. Each key maps to a fixed SQL predicate in `GuestReadService.SegmentPredicates`; an unknown key becomes `1 = 0`, so no user text is ever concatenated into SQL. When adding a count that should open a list, add the key in all three places.

4.4 Transactional outbox

Domain events are saved as `OutboxMessages` rows in the same transaction as the change (`OutboxSaveChangesInterceptor`), so an event cannot be lost if the API stops after committing. `OutboxRelayWorker` relays them to an in-process `Channel<T>` (`InProcessOutboxEventChannel`) read by `EscalationWorker`. `IOutboxEventChannel` is the seam for moving to a message broker later. Only

`RiskFlagRaisedEvent` and `UrgentCaseResolvedEvent` have a consumer; `GuestRegisteredEvent` and `FollowUpScheduledEvent` are relayed and ignored.

4.5 Read models refreshed by workers

Dashboards and some reports read denormalised tables rather than grouping over all history on each request.

Table	Maintained by	Read by
<code>UrgentCases_ReadModel</code>	<code>EscalationWorker</code> (and <code>GuestAnonymiser</code> on anonymisation)	Active urgent-case list, dashboard urgent banner
<code>DashboardSnapshots_ReadModel</code>	<code>ReportMaterializerWorker</code> , every 5 minutes	CMHW and Hub Manager dashboard counts, pathway distribution, monthly stats, clinical complexity, demographics, data quality cards
<code>PathwayReportAggregates_ReadModel</code>	<code>ReportMaterializerWorker</code> , every 5 minutes	Nothing since 30 Sep 2026 (referral categories are no longer reported); safe to retire

`ReportMaterializerWorker` loops over the rows in the `Hubs` table. A hub with staff and guests but no `Hubs` row gets no snapshot, and its dashboard shows zeros. There is no screen for creating hubs; rows come from the seeder or SQL (document 06 has the SQL).

4.6 Permissions as claims, roles editable by admins

- The permission catalogue is fixed in code (`Permissions.All`, 35 permissions). Each permission is registered as its own authorisation policy in `Program.cs` and applied with `[Authorize(Policy = Permissions.X.Y)]`.
- Roles are data: `ApplicationRole` rows with permissions stored as role claims of type `permission`. Admins create and edit roles on the Roles & Permissions screen (`/hub-workers/roles`, backed by the `/admin/roles` API).
- At sign-in, `TokenService` flattens all of the user's role permissions onto the JWT. `PermissionAuthorizationHandler` only checks the token, so there is no database lookup per request. The consequence is that role changes, and deactivation, take effect only when the user next signs in (tokens last 8 hours).
- `IdentitySeeder` runs at every API start. It creates the built-in roles `Cmhw`, `Cpn`, `HubManager` and `Admin` if missing, and **adds back any default permission missing from a built-in role**. It never removes permissions. Removing a default permission from a built-in role therefore lasts only until the next deploy; use a custom role instead.

Built-in role	Default permissions
<code>Cmhw</code>	CMHW dashboard; guest view, register, edit; demographics, clinical and pathway view and edit; notes view and add; add contacts; follow-ups view and manage; urgent cases view; reports view; documents view, upload, edit
<code>Cpn</code>	Everything <code>Cmhw</code> has, plus <code>guests.contacts.cpn</code> (log CPN contacts and assessments)
<code>HubManager</code>	Everything <code>Cmhw</code> has, plus Hub Manager dashboard, <code>mdt.manage</code> , <code>reports.export</code> , documents delete and restore, <code>settings.view</code> , <code>guests.audit.view</code> , <code>guests.export</code>
<code>Admin</code>	Every permission, including <code>guests.erase</code> , <code>documents.purge</code> , <code>settings.manage</code> , <code>settings.lookups.manage</code> , <code>admin.manageusers</code> , <code>admin.manageroles</code>

4.7 Audit interceptor and read-logging middleware

- `AuditSaveChangesInterceptor` adds an `AuditEvent` for every create, update or delete of 26 audited entity types, in the same transaction. For updates it records which fields changed, never their values, so the log does not duplicate personal data.
- `AuditReadLoggingMiddleware` adds a `Read` event after every successful (2xx) GET whose path starts `/guests/{guid}` or `/urgent-cases/{guid}`.
- Reads that cannot be tied to a guest from the URL (episode records by episode id, document downloads, subject-access exports, anonymisation) are written explicitly through `IAuditTrail`.
- Background writes are attributed to an empty staff id, shown as "System".
- `AuditDescriptions.Describe` turns the stored action and entity name into plain English ("Opened guest record", "Recorded a contact") for the Hub Manager's staff activity feed and the guest's Access Log tab. The stored rows are unchanged.

4.8 Anonymise, never delete

Guest records are never hard-deleted. `POST /guests/{id}/anonymise` (permission `guests.erase`, Admin only by default) calls `Guest.Anonymise()`, which replaces the name with "Anonymised" and "Guest {number}", keeps only the year of birth, clears gender, phone, email, address, postcode and legacy reference, and sets `IsAnonymised` and `IsDeleted`. The handler (`AnonymiseGuestCommandHandler` in `GuestComplianceQueries.cs`) also anonymises the demographics row, moves the guest's documents to the recycle bin, scrubs the name from `UrgentCases_ReadModel`, and writes the reason to the audit log. It is refused while the guest is flagged urgent. Staff accounts are likewise only deactivated (`IsActive = false`), because their ids appear throughout the audit trail.

4.9 Other conventions

- Enums are serialised as names ("`Active`"), both in REST responses and SignalR payloads.
- Built-in dropdown options are seeded by `LookupSeeder` and can be relabelled, reordered or deactivated but not deleted.
- Admin-defined extra fields (`CustomFieldDefinition`, `CustomFieldValue`) can be added to configurable forms; values inherit the view and edit permission of the record they belong to.
- Clinical history is append-only: risk assessments are versioned, pathway changes and CMHW assignments are separate history rows, and document versions are immutable.

5. Data model overview

All tables are in the `Emhip` database, created by EF Core migrations. Ids are GUIDs.

Group	Tables (entity)	Notes
Guest core	<code>Guests</code> , <code>GuestDemographics</code> , <code>GuestClinicalProfiles</code> , <code>InitialConversationRecords</code>	Demographics, clinical profile and initial conversation are one-to-one with the guest, in their own tables
Contacts and casework	<code>Contacts</code> , <code>CaseworkNotes</code> , <code>CaseworkSessions</code> , <code>FollowUps</code> , <code>GuestActions</code> , <code>Notes</code> , <code>CarePlans</code> , <code>CarePlanGoals</code>	Casework notes are SBAR, Draft then Submitted; follow-up status is Scheduled, Overdue, Completed or Cancelled

Group	Tables (entity)	Notes
Risk and urgent care	<code>RiskAssessments</code> , <code>UrgentEpisodes</code>	Risk assessments are versioned and append-only; an urgent episode runs from flag to resolution
Outcomes and CPN	<code>DialogAssessments</code> , <code>CpnInitialAssessments</code> , <code>CpnRiskDomainRatings</code> , <code>MdtQueueItems</code>	MDT items are CPN referrals, initial reviews or discussion requests
Pathways and allocation	<code>PathwayReferrals</code> , <code>PathwayChanges</code> , <code>CaseLoadAssignments</code>	Changes and assignments are history rows
Documents	<code>Documents</code> , <code>DocumentVersions</code>	Soft delete, restore, purge after retention date, check-out and check-in
Configuration	<code>Hubs</code> , <code>AppSettings</code> , <code>LookupItems</code> , <code>EmailTemplates</code> , <code>CustomFieldDefinitions</code> , <code>CustomFieldValues</code>	<code>AppSettings</code> stores only overrides of catalogue defaults
Identity	<code>AspNetUsers</code> , <code>AspNetRoles</code> , <code>AspNetUserRoles</code> , <code>AspNetRoleClaims</code> and the other Identity tables	<code>AspNetUsers</code> adds <code>DisplayName</code> , <code>HubId</code> , <code>IsActive</code> ; roles add <code>Description</code>
Infrastructure	<code>AuditEvents</code> , <code>OutboxMessages</code> , <code>ExportRecords</code>	Nothing ever purges them
Read models	<code>UrgentCases_ReadModel</code> , <code>DashboardSnapshots_ReadModel</code> , <code>PathwayReportAggregates_ReadModel</code>	Written only by workers (and the anonymiser)

Staff-id columns such as `Guest.AssignedCmhwId` and `Contact.CreatedByStaffId` are plain GUIDs with no foreign key to `AspNetUsers` . `Guests.HubId` has no foreign key to `Hubs` either.

6. API overview

Public URLs are prefixed with `/api` (the `client` nginx strips it). Every controller carries `[Authorize]` except `AuthController` and `InternalNotificationsController` ; `/health` is also open.

Controller	Route prefix	Purpose	Main permission
<code>AuthController</code>	<code>/auth</code>	Login, forgot password, reset password, change password, current user	None (login, forgot and reset are rate-limited); see section 15.1
<code>AdminUsersController</code>	<code>/admin/users</code>	List, create, update, deactivate staff; admin password reset	<code>admin.manageusers</code>
<code>AdminRolesController</code>	<code>/admin/roles</code>	Roles and the grouped permission catalogue	<code>admin.manageroles</code>
<code>MigrationController</code>	<code>/admin/migration</code>	CSV template and legacy guest import (dry run by default)	<code>admin.manageusers</code>
<code>GuestsController</code>	<code>/guests</code>	Guest list, search, registration, every workspace tab, notes, casework notes, CPN record, care plan, access log, export, anonymise	<code>guests.view</code> plus a per-action permission

Controller	Route prefix	Purpose	Main permission
ContactsController	/contacts	Hub-wide contact history, per-guest summary, stat tiles	guests.view
FollowUpsController	/followups	Scheduled contacts queue; mark complete	followups.view, followups.manage
UrgentCasesController	/urgent-cases	Active and resolved cases, episode records and export, escalate to CMHT, resolve	urgentcases.view; guests.clinical.edit to escalate or resolve
MdtController	/mdt	MDT queue: confirm CPN, decline, mark discussed	mdt.manage
DashboardsController	/dashboards	CMHW dashboard, Hub Manager overview, guests-seen card	dashboard.cmhw.view, dashboard.hubmanager.view
ReportsController	/reports	Pathways, DIALOG outcomes and trend, pathway analytics, caseload, data quality, CPN activity, contacts, referral sources, export history, CSV and Excel export	reports.view; reports.export for exports
DocumentsController	/documents	Register, stats, upload, versions, download, delete, restore, purge, check-out, check-in	documents.*
SettingsController	/settings	Settings editor, public settings, storage test, email test	settings.view, settings.manage; /settings/public for any signed-in user
EmailTemplatesController	/email-templates	List, edit, reset and preview templates	settings.view; settings.manage to change
LookupsController	/lookups	Dropdown options	Signed in to read; settings.lookups.manage to change
CustomFieldsController	/custom-fields	Extra field definitions and values	Signed in to read; settings.manage for definitions
InternalNotificationsController	/internal-urgent-cases	Worker-to-API SignalR relay	X-Internal-Secret header only
Minimal API	/health	Liveness probe; does not touch the database	None
SignalR hub	/hubs/urgent-cases	Live urgent-case events	urgentcases.view

The Excel export (`GET /reports/export.xlsx`) is built by `ExcelWorkbookBuilder` with ClosedXML and has seven sheets: Summary, Demographics, Referral sources, Pathways, Caseload, DIALOG outcomes and Data quality. Sheet names must stay in step with `WORKBOOK_SHEETS` in `client/src/app/features/reports/report-meta.ts`. The optional cohort parameters (`ethnicity`, `gender`, `countryOfOrigin`, `ageMin`, `ageMax`) are parsed by `ReportCohortFilter` and apply to the DIALOG outcomes sheet, the DIALOG outcomes tab and the DIALOG trend. The CSV export (`GET /reports/export`) streams one row per guest registered in the period (G-number, name, pathway, status, registration date,

demographics, referral source and type) as the rows are read. Pathway names come from `GuestPathwayLabels` (server) and `CLINICAL_PATHWAY_OPTIONS` (client), which must stay in step. Both exports are recorded in `ExportRecords`.

Swagger UI is served at `/swagger` only when `ASPNETCORE_ENVIRONMENT` is `Development` (local `dotnet run`). It is not available in the Docker containers, whose environment is `Docker`.

7. Front-end structure

The client is an Angular 22 app using standalone components, signals and lazy-loaded routes (`loadComponent`). There are no NgModules and no state library; state lives in services and component signals. Styles are SCSS with Figma-derived tokens in `client/src/styles`. Fonts load from Google Fonts. Production builds swap `environment.ts` (API at `http://localhost:5299`) for `environment.prod.ts` (API at `/api`, hub at `/hubs/urgent-cases`).

7.1 Routes

All routes except the three sign-in pages sit inside the shell (`AppShellComponent`) and require a valid session (`authGuard`). Routes with a permission also use `permissionGuard`, which redirects to `/dashboard` when the permission is missing.

Route	Screen	Permission
<code>/login</code> , <code>/forgot-password</code> , <code>/reset-password</code>	Sign-in and password reset	Public
<code>/dashboard</code>	CMHW dashboard, or Hub Manager overview if the user has <code>dashboard.hubmanager.view</code>	Signed in
<code>/guests</code>	Guest list with filters and segment drill-throughs	<code>guests.view</code>
<code>/guests/new</code>	Register New Guest (demographics, initial conversation, DIALOG, pathway, review)	<code>guests.register</code>
<code>/guests/:guestId</code>	Guest workspace (12 tabs, plus Access Log with <code>guests.audit.view</code>)	<code>guests.view</code>
<code>/followups</code>	Scheduled contacts queue; linked from the Hub Manager dashboard, not in the menu	<code>followups.view</code>
<code>/urgent-cases</code>	Urgent Cases list, details drawer and episode record	<code>urgentcases.view</code>
<code>/mdt-queue</code>	MDT queue	<code>mdt.manage</code>
<code>/contact-history</code>	Hub-wide contact history	<code>guests.view</code>
<code>/reports</code>	Reports tabs and exports	<code>reports.view</code>
<code>/documents</code>	Hub-wide document register; hidden from the menu but still reachable	<code>documents.view</code>
<code>/settings</code>	Settings, email templates, option lists, custom fields, data migration	<code>settings.view</code>

Route	Screen	Permission
/hub-workers	Staff accounts	admin.manag eusers
/hub-workers/roles	Roles and permissions	admin.manag eroles

Unknown paths inside the shell redirect to `/dashboard`; anything else redirects to `/login`.

7.2 Core services (client/src/app/core)

- `auth.service.ts`: signs in, stores the session (token, expiry, user, roles, permissions) in `localStorage` under `emhip_session`, and exposes `hasPermission` and `hasAnyPermission`.
- `auth.interceptor.ts`: adds the bearer token only to requests for EMHIP's own API; on a 401 it signs the user out and goes to `/login?reason=expired`.
- `auth.guard.ts`, `permission.guard.ts`: route guards described above.
- `idle-timeout.service.ts`: signs the user out after `security.sessionIdleMinutes` without activity, with a 60-second warning.
- `urgent-cases-hub.service.ts`: the SignalR connection, exposing `latestEscalation` and `latestResolution` signals.
- One API service per controller: `guests-api`, `contacts-api`, `follow-ups-api`, `urgent-cases-api`, `mdt-api`, `dashboards-api`, `reports-api`, `documents-api`, `settings-api` (also lookups and email templates), `custom-fields-api`, `admin-api`, `migration-api`.
- `api-models.ts`: TypeScript copies of the server DTOs. Keep it in step by hand when a DTO changes.
- `permissions.ts`, `guest-segments.ts`, `demographic-options.ts`: client mirrors of server constants.

7.3 Shared components (client/src/app/shared)

`guest-picker` and `staff-picker` (searchable pickers), `staff-directory.service`, `demographic-filters` (the four demographic filters used by the guest list and reports), `custom-fields` (renders admin-defined fields on a form) and `emhip-logo`. The design system folder holds `icon.component.ts` and 51 generated icons.

7.4 How permissions gate the UI

- **Routes**: `data.permission` on the route plus `permissionGuard`.
- **Menu**: each `NavItem` in `app-shell.component.ts` lists the permissions that show it (any one is enough). A `hidden: true` flag keeps an item out of the menu while its route still works; Documents uses it at the customer's request.
- **Buttons and tabs**: components call `auth.hasPermission(...)` in more than 50 places. A few buttons still show for everyone and rely on the server to refuse; document 08 lists them.
- The server is the real enforcement. The client only reads the permission list returned at sign-in.

7.5 Reload after a deploy

Each deploy replaces the hashed lazy-loaded bundles, so a tab left open across a deploy cannot load screens it has not visited. `app.config.ts` registers `withNavigationErrorHandler(reloadOnStaleChunk)`: when navigation fails with a chunk-load error it reloads the browser straight to the target URL, once per URL (tracked in `sessionStorage` under `emhip_chunk_reload`). A second failure in a row is left to surface as a real error.

8. Authentication and security

- **Accounts:** ASP.NET Core Identity with local accounts (`ApplicationUser`, `ApplicationRole`). There is no self-registration and no single sign-on; admins create accounts on the Hub Workers screen.
- **Tokens:** `POST /auth/login` returns an HS256 JWT (issuer `Emhip.Api`, audience `Emhip.Client`, lifetime `Jwt:ExpiryMinutes`, default 480 minutes, clock skew 1 minute). It carries the staff id, email, display name, hub id, roles and permissions. There is no refresh token; the user signs in again when it expires.
- **Password rules:** at least 10 characters, with an upper-case letter, a lower-case letter and a digit (Identity defaults, with the length raised to 10 and the symbol requirement turned off). Emails must be unique.
- **Lockout:** 5 failed attempts lock the account for 15 minutes. Login gives the same answer for unknown, inactive, locked and wrong-password accounts.
- **Rate limiting:** the `auth` policy allows 10 requests per minute per client IP on login, forgot password and reset password, returning 429 when exceeded. See section 15 for a caveat about the IP address behind the proxies.
- **Password reset:** forgot password emails a link to `{Frontend:BaseUrl}/reset-password` containing an Identity reset token, and always returns 204. Admins can also set a new password on the Hub Workers screen. The API has `POST /auth/change-password`, but the app has no screen that calls it.
- **Idle sign-out:** the app signs the user out after 30 minutes without activity by default (`security.sessionIdleMinutes`, 0 disables). This is client-side; the token itself stays valid until it expires.
- **Security headers (API):** `SecurityHeadersMiddleware` sets `X-Content-Type-Options`, `X-Frame-Options: DENY`, `Referrer-Policy: no-referrer`, `Permissions-Policy`, `Cross-Origin-Opener-Policy`, `Cross-Origin-Resource-Policy`, a default-src 'none' Content-Security-Policy (relaxed for Swagger) and `Cache-Control: no-store` on everything except `/health`. HTTPS redirection is on in every environment except `Docker`; HSTS is on except in `Docker` and `Development`.
- **Security headers (SPA):** `client/nginx.conf` sets HSTS, the same frame, type and referrer headers, and a CSP that allows only EMHIP's own scripts plus Google Fonts for styles and fonts.
- **Service-to-service:** the workers authenticate to the internal notification endpoint with `Internal:SharedSecret`, compared in constant time.
- **Hub scoping:** list, search, report and dashboard queries filter by the caller's hub. See section 15 for the guest-by-id endpoints, which do not.

9. Background workers

All five run in the `workers` container, start immediately when the container starts, and then repeat on their interval. Errors are logged and the loop continues.

Worker	What it does	How often
<code>OutboxRelayWorker</code>	Reads up to 100 unprocessed <code>OutboxMessages</code> (oldest first), publishes them to the in-process channel and marks them processed; failures record the error and an attempt count	Every 2 seconds
<code>EscalationWorker</code>	For a raised risk flag: upserts <code>UrgentCases_ReadModel</code> , pushes the SignalR event through the API, and emails the assigned worker. For a resolution: deactivates the read-model row and pushes the resolved event	As events arrive
<code>FollowUpSchedulerWorker</code>	Marks Scheduled follow-ups with a due date before today (UTC) as Overdue; when a sweep changes any, emails every assignee who has overdue items one digest listing up to 20 of them	Every 15 minutes

Worker	What it does	How often
ReportMaterializerWorker	For each row in <code>Hubs</code> , recomputes the dashboard snapshot and the monthly pathway aggregates	Every 5 minutes
EngagementStatusWorker	Moves Active, non-urgent guests with no activity for <code>clinical.inactivityDays</code> (default 90; registration date if never active) to OnHold	Every 6 hours

The workers share the API's Infrastructure registrations (`AddInfrastructure`) but not MediatR. If the `workers` container is down, new urgent cases do not reach the Urgent Cases list; the outbox rows wait and are relayed when it starts again.

10. Configuration reference

Configuration comes from `appsettings.json`, environment variables (with `__` for `:`, for example `ConnectionStrings__Emhip`) and, in Docker, from values Compose interpolates from `.env`. Real secret values are never in the repository: set them in `.env` on the server.

10.1 Compose variables (`.env`)

Variable	Purpose	Default
<code>MSSQL_SA_PASSWORD</code>	SQL Server <code>sa</code> password; used in every connection string and by <code>backup.sh</code>	None, required; set in <code>.env</code>
<code>JWT_KEY</code>	Becomes <code>Jwt__Key</code> , the token signing key; 32 characters or more	None, required; set in <code>.env</code>
<code>INTERNAL_SHARE_D_SECRET</code>	Becomes <code>Internal__SharedSecret</code> on the API and workers	None, required; set in <code>.env</code>
<code>BOOTSTRAP_ADMIN_EMAIL</code>	Becomes <code>Bootstrap__AdminEmail</code>	<code>admin@emhip.local</code>
<code>BOOTSTRAP_ADMIN_PASSWORD</code>	Becomes <code>Bootstrap__AdminPassword</code>	None, required; set in <code>.env</code>
<code>PUBLIC_ORIGIN</code>	Production only; becomes <code>Cors__AllowedOrigins__0</code> and <code>Frontend__BaseUrl</code> on the API	None, required in production; set in <code>.env</code>
<code>MSSQL_PID</code>	Production only; SQL Server edition	<code>Express</code> (the development file hard-codes <code>Developer</code>)

10.2 API settings

Key	Purpose	Default
<code>ConnectionStrings:Emhip</code>	SQL Server connection	<code>(local)</code> trusted connection in <code>appsettings.json</code> ; Compose builds it from <code>MSSQL_SA_PASSWORD</code>
<code>ApplyMigrationsOnStartup</code>	Apply migrations and run the Identity, lookup and email-template seeders at startup	<code>false</code> ; <code>true</code> in <code>docker-compose.yml</code> , so on in production

Key	Purpose	Default
<code>ASPNETCORE_ENVIRONMENT</code>	<code>Development</code> enables Swagger; <code>Docker</code> turns off HTTPS redirection and HSTS	<code>Docker</code> in Compose; <code>Development</code> in <code>launchSettings.json</code>
<code>ASPNETCORE_URLS</code>	Listen address	<code>http://+:8080</code> in the Dockerfile; <code>http://localhost:5299</code> locally (from <code>launchSettings.json</code>)
<code>Cors:AllowedOrigins</code>	Allowed browser origins	<code>http://localhost:4200</code> ; <code>http://localhost:8080</code> in Compose; <code>PUBLIC_ORIGIN</code> in production
<code>Frontend:BaseUrl</code>	Base of links in emails sent by the API	<code>http://localhost:4200</code> ; <code>http://localhost:8080</code> in Compose; <code>PUBLIC_ORIGIN</code> in production
<code>Jwt:Key</code>	Signing key	Placeholder in <code>appsettings.json</code> ; set from <code>JWT_KEY</code>
<code>Jwt:Issuer</code> , <code>Jwt:Audience</code>	Token issuer and audience	<code>Emhip.Api</code> , <code>Emhip.Client</code>
<code>Jwt:ExpiryMinutes</code>	Token lifetime	<code>480</code>
<code>Internal:SharedSecret</code>	Secret the workers must present	Placeholder; set from <code>INTERNAL_SHARED_SECRET</code>
<code>Bootstrap:AdminEmail</code>	First admin's email, used only when no users exist	<code>admin@emhip.local</code>
<code>Bootstrap:AdminPassword</code>	First admin's password, used only when no users exist	Placeholder; set from <code>BOOTSTRAP_ADMIN_PASSWORD</code>
<code>Bootstrap:AdminHubId</code>	Hub id given to the first admin	<code>22222222-2222-2222-2222-222222222222</code>
<code>Logging:LogLevel</code>	Log levels	<code>Information</code> ; <code>Microsoft.AspNetCore</code> at <code>Warning</code>

10.3 Workers, seeder and SQL Server settings

Key	Purpose	Default
<code>ConnectionStrings:Emhip</code> (workers)	SQL Server connection	As for the API
<code>Api:BaseUrl</code> (workers)	Where the workers call the internal notification endpoint	<code>https://localhost:5001/</code> in <code>appsettings.json</code> ; <code>http://api:8080/</code> in Compose
<code>Internal:SharedSecret</code> (workers)	Must match the API's value	Placeholder; set from <code>INTERNAL_SHARED_SECRET</code>
<code>Frontend:BaseUrl</code> (workers)	Base of links in emails sent by the workers	<code>http://localhost:8080</code> in Compose; <code>PUBLIC_ORIGIN</code> in production
<code>EMHIP_CONNECTION</code> (seeder)	Connection string when <code>--connection</code> is not given	Set by Compose for the <code>seeder</code> profile

Key	Purpose	Default
ACCEPT_EULA (SQL Server)	Accepts the SQL Server licence	Y
MSSQL_MEMORY_LIMIT_MB (SQL Server)	Memory cap	4096 in production

10.4 Runtime settings (Settings page)

These are stored in the `AppSettings` table and edited on the Settings page; the catalogue with labels and defaults is `Emhip.Application/Settings/SettingsCatalog.cs`. Each process caches settings for 5 minutes; a save clears the API's cache at once, so the workers can take up to 5 minutes to see a change. Values marked secret are never returned to the browser but are stored as plain text in the table.

Key	Default	Used by
general.organisationName	EMHIP	Emails, Excel export, Documents page
general.supportEmail	Empty	Email templates
general.dateFormat	dd MMM yyyy	Nothing (see section 15)
documents.storage.provider	Local	New uploads
documents.storage.local.root	/var/emhip/documents	Local storage
documents.storage.s3.*	Region eu-west-2 ; path-style true	S3 and S3-compatible storage (bucket, keys, service URL)
documents.storage.azure.*	Container emhip-documents	Azure Blob storage (connection string, container)
documents.storage.gcp.*	None	Google Cloud Storage (bucket, service-account JSON)
documents.upload.maxFileSizeMb	25	Upload checks, client and server
documents.upload.allowedExtensions	pdf,doc,docx,xls,xlsx,png,jpg,jpeg,txt,csv,rtf,odt	Upload checks, client and server
documents.retentionYears	7	Default retention date on new documents
clinical.urgentResponseHours	72	Episode record deadline and urgent email; not the Urgent Cases list
clinical.inactivityDays	90	EngagementStatusWorker (automatic move to Inactive)
clinical.followUpDefaultDays	14	Nothing (see section 15)
clinical.dialogReviewWeeks	12	Nothing (see section 15)
ui.guestListPageSize	50	Nothing (see section 15)
security.sessionIdleMinutes	30	Idle sign-out

Key	Default	Used by
<code>compliance.recordRetention</code> Years	20	Data Quality "past retention" check and segment
<code>email.provider</code>	None	Email; <code>SmtP</code> , <code>AwsSes</code> or <code>Mailgun</code>
<code>email.fromAddress</code> , <code>email.fromName</code> , <code>email.replyTo</code>	From name <code>EMHIP Portal</code>	Every email
<code>email.smtp.*</code>	Port <code>587</code> , security <code>StartTls</code>	SMTP host, port, username, password, security
<code>email.ses.*</code>	Region <code>eu-west-2</code>	SES region and keys
<code>email.mailgun.*</code>	Region <code>US</code>	Mailgun domain, API key, region
<code>email.notifyOnUrgentCase</code>	true	<code>EscalationWorker</code>
<code>email.notifyOnOverdueFollowUps</code>	true	<code>FollowUpSchedulerWorker</code>

10.5 Client build settings

`client/src/environments/environment.ts` (development) and `environment.prod.ts` (production builds, swapped in by `angular.json`) hold `apiBaseUrl` and `signalRHubUrl`. The production build also enforces size budgets: initial bundle warning at 500 kB and error at 1 MB; component styles warning at 8 kB and error at 24 kB.

11. Local development

11.1 Prerequisites

- .NET 10 SDK (10.0.102 was used).
- Node.js 24 through nvm for the Angular CLI. Angular 22 needs Node 22.22 or later; the client Dockerfile builds with `node:22-alpine`. Without nvm switching, a shell that defaults to an older Node fails to build.
- npm 10 (the `packageManager` field pins npm 10.9.8).
- The EF Core tool: `dotnet tool install --global dotnet-ef` (version 10.x).
- A SQL Server instance. On macOS or Linux, run SQL Server 2022 in Docker; on Windows, a local instance works with the default `(local)` connection string.
- Docker with the Compose plugin, if you want the whole stack.

11.2 Option A: the whole stack in Docker

Copy `.env.example` to `.env`, set real values for every variable, then run `docker compose up --build` from the repository root. The app is at `http://localhost:8080`, the API at `http://localhost:5299` (no Swagger in this mode) and SQL Server at `localhost:1433`. Migrations and seeding run automatically.

11.3 Option B: API, workers and client on the host

Start SQL Server (skip if you already have one):

```
docker run -d --name emhip-sql -e ACCEPT_EULA=Y -e MSSQL_SA_PASSWORD='choose-a-strong-password' \
-p 1433:1433 mcr.microsoft.com/mssql/server:2022-latest
```

Run the API. `ApplyMigrationsOnStartup=true` creates the database, the built-in roles, the first admin, the option lists and the email templates:

```
export ConnectionStrings__Emhip='Server=localhost,1433;Database=Emhip;User
Id=sa;Password=choose-a-strong-password;TrustServerCertificate=True;'
export ApplyMigrationsOnStartup=true
export Bootstrap__AdminPassword='choose-an-admin-password'
dotnet run --project src/Emhip.Api
```

The API listens on `http://localhost:5299` with Swagger at `http://localhost:5299/swagger`. Sign in as `admin@emhip.local` with the password you chose.

Warning: the admin password must meet the password rules in section 8 (10 characters with upper case, lower case and a digit). If it does not, `IdentitySeeder` fails silently and no admin is created; fix the value and restart the API.

Run the workers in a second terminal. They must be pointed at port 5299, because their default `Api:BaseUrl` is `https://localhost:5001/`:

```
export ConnectionStrings__Emhip='Server=localhost,1433;Database=Emhip;User
Id=sa;Password=choose-a-strong-password;TrustServerCertificate=True;'
export Api__BaseUrl='http://localhost:5299/'
dotnet run --project src/Emhip.Workers
```

Run the client in a third terminal:

```
nvm use 24
cd client
npm ci
npx ng serve
```

Open `http://localhost:4200`. The development environment calls the API at `http://localhost:5299`, and the API's default CORS origin is `http://localhost:4200`.

Note: the placeholder values of `Jwt:Key` and `Internal:SharedSecret` in `appsettings.json` let the local API and workers work together. Never use them outside a developer machine.

11.4 Seeding test data

`tools/Emhip.Seeder` creates hubs with random ids, eight staff per hub (the first a Hub Manager, the rest CMHWs, all with one temporary password that the seeder prints), and guests with contacts, notes, risk assessments, follow-ups, referrals and audit events.

```
dotnet run --project tools/Emhip.Seeder -- --connection "$ConnectionString__Emhip" --hubs 1
--guests 2000
```

Options are `--hubs` (default 3), `--staff-per-hub` (8), `--guests` per hub (20000) and `--batch-size` (5000). With Docker: `docker compose --profile seed run --rm seeder --guests 2000 --hubs 1`. The first

admin's hub (`Bootstrap.AdminHubId`) is not one of the seeded hubs; to see the seeded data as admin, look up a hub id with `SELECT Id, Code FROM Hubs` and set it as the admin's Hub ID on the Hub Workers screen, then sign in again.

Warning: the seeder writes directly with `SqlBulkCopy` and creates accounts with a shared password. Never point it at production.

11.5 Building and testing

- Back end: `dotnet build Emhip.slnx` and `dotnet test Emhip.slnx` from the repository root.
- Front end: from `client`, `npx ng build` (production configuration by default) and `npx ng test --watch=false` (Vitest).
- Before a push to `main`, run both test suites and a production client build: a push to `main` deploys to production with no further checks (document 06).

12. Database and migrations

- EF Core 10 with SQL Server. `EmhipDbContext` derives from `IdentityDbContext<ApplicationUser, ApplicationRole, Guid>`; entity configurations are in `Persistence/Configurations`.
- There are 14 migrations in `src/Emhip.Infrastructure/Persistence/Migrations`, from `InitialCreate` (13 July 2026) to `AddEpisodeRecordNoteAttachmentsAndAnonymisation` (9 September 2026). On 30 September 2026 `dotnet ef migrations has-pending-model-changes` reported no pending changes.
- `EmhipDbContextFactory` is the design-time factory. It hard-codes a `(local)` trusted connection, which is fine for generating migrations but not for applying them from macOS or Linux: pass `--connection` to `database update`.

12.1 Adding a migration

1. Change the entity and its configuration.
2. Generate the migration from the repository root:
 - `dotnet ef migrations add AddSomething --project src/Emhip.Infrastructure --startup-project src/Emhip.Infrastructure --output-dir Persistence/Migrations`
3. Read the generated `Up` and `Down` methods. Make sure the change is safe for existing rows (defaults for new non-null columns, no silent data loss).
4. Update any hand-written SQL that uses the changed columns: `GuestReadService` (list, count, `SegmentPredicates`) and `FollowUpReadService`. The compiler does not check these strings.
5. Apply it to your local database, either by starting the API with `ApplyMigrationsOnStartup=true` or with `dotnet ef database update --project src/Emhip.Infrastructure --startup-project src/Emhip.Infrastructure --connection "$ConnectionString__Emhip"`.
6. Test against SQL Server, then commit the migration, its designer file and the updated `EmhipDbContextModelSnapshot.cs` together.

12.2 How migrations reach production

Production runs `docker-compose.yml` with `docker-compose.prod.yml` layered on top, and `docker-compose.yml` sets `ApplyMigrationsOnStartup=true`. When the new API container starts it calls `Database.MigrateAsync()`, then `IdentitySeeder`, `LookupSeeder` and `EmailTemplateSeeder`. `deploy.sh` takes a full database backup before every deploy. If a migration fails, the API does not start and Docker

restarts it in a loop; the fix is a corrected migration pushed to `main`, or a restore of the pre-deploy backup (document 06).

Rules for production migrations:

- Keep them additive and backwards compatible where possible, because the only rollback path for schema is a restore.
- Never edit or delete a migration that has been deployed; add a new one.
- Large data changes run inside API startup; if one would take minutes, run it separately and keep the migration small.

13. Testing

Suite	What it covers	Size
<code>tests/Emhip.UnitTests</code> (xUnit)	Guest engagement status, risk assessments, urgent episodes, casework notes, CPN assessments, documents, custom fields, anonymisation, registration validation, keyset cursors, email template rendering, the report cohort filter, urgent episode record text	74 tests, all passing on 30 September 2026
<code>tests/Emhip.IntegrationTests</code>	Starts the API with <code>WebApplicationFactory<Program></code> and calls <code>/health</code>	1 test, passing
<code>client (ng test, Vitest)</code>	<code>app.spec.ts</code> checks the root component is created	1 spec

Gaps:

- Nothing runs against a real SQL Server. The hand-written Dapper SQL, the segment predicates and every EF Core query translation are untested until they reach production. Several production 500 errors have come from this (for example commits `7eece63` and `4b3a264`, both guest-list SQL).
- No API tests for permissions, hub scoping or validation.
- No browser end-to-end tests.
- No continuous integration: no workflow runs the tests before a push to `main` deploys.

14. Working practices

- `main` is production. Work on a branch, run the checks in section 11.5, then merge.
- Keep server and client constants in step by hand: `Permissions.cs` and `permissions.ts`, `GuestSegments.cs` and `guest-segments.ts`, `SettingsCatalog.Keys` and `SettingKeys`, `LookupSeeder.Categories` and `LookupCategories`, DTOs and `api-models.ts`, Excel sheet names and `WORKBOOK_SHEETS`.
- When the customer asks to remove a menu item, hide it with `hidden: true` on the `NavItem` and keep the route and component.
- Extend `IAuditTrail` and the access log rather than adding ad-hoc logging of guest data.
- Keep anonymisation as the only erasure path for guest records.

15. Known technical debt and recommendations

Every item below was checked in the code on 30 September 2026. Document 08 covers the user-facing effect of several of them.

15.1 Security and data protection

- **Guest-by-id endpoints are not hub-scoped.** The workspace queries (`GetGuestOverviewQuery`, `GetGuestDemographicsQuery`, `GetGuestClinicalQuery` and the other tab queries in `GetGuestWorkspaceQueries.cs`) take only a guest id, so a user who knows another hub's guest id could read it. Lists, search, reports, the access log and export are scoped. With one hub in production this is low risk; fix it before adding a second hub, for example with a shared "guest belongs to caller's hub" check.
- **The internal endpoint is reachable from the internet.** `client/nginx.conf` forwards everything under `/api/` to the API, so `https://emhip.brainshub.co.uk/api/internal/urgent-cases/notify` is public and protected only by `INTERNAL_SHARED_SECRET`. Add `location /api/internal/ { return 404; }` to `client/nginx.conf` (the workers call the API directly on the Docker network, not through nginx).
- **Login rate limiting is shared by everyone.** The limiter partitions on `RemoteIpAddress`, and there is no forwarded-headers middleware, so behind the two nginx proxies every request appears to come from the `client` container. The 10-per-minute limit therefore applies to the whole organisation at once. Add `UseForwardedHeaders` with the Docker network as a known proxy, or partition on the submitted email address.
- **Token changes are not immediate.** Deactivating a user or changing their role takes effect only at their next sign-in; an existing token stays valid for up to 8 hours. Consider a shorter lifetime with refresh, or a security-stamp check.
- **Data Protection keys are not persisted.** Nothing configures `AddDataProtection().PersistKeysTo...`, so the keys that protect password-reset tokens live inside the API container and are replaced when it is recreated. Reset links sent before a deploy stop working. Persist the keys to a volume.
- **No change-password screen.** `POST /auth/change-password` exists but no screen calls it. Staff cannot change the temporary password an admin gave them, and the account-created email includes that temporary password in plain text. Add a change-password page and force a change at first sign-in.
- **Two auth endpoints lack `[Authorize]`.** `GET /auth/me` answers anonymous callers with an empty user (200), and `POST /auth/change-password` relies on the handler finding no user. Neither leaks data, but add `[Authorize]` to both.
- **Settings secrets are stored in plain text** in `AppSettings` (SMTP password, SES and Mailgun keys, storage keys). Encrypt them with ASP.NET Core Data Protection once the keys are persisted, or move them to `.env`.
- **No multi-factor authentication**, and no encryption at rest in the application (see document 09).

15.2 Behaviour and correctness

- **Save Draft on Register New Guest does not persist.** `RegisterGuestComponent.saveDraft()` only records the time for the "Draft saved" label; leaving the page loses the data.
- **Two urgent-response windows.** The Urgent Cases list and drawer use `WINDOW_HOURS = 72` in `urgent-cases.component.ts`; the episode record and the urgent email use `clinical.urgentResponseHours`. Read the setting on the client too (`SettingsApiService.urgentResponseHours` already exists and is unused).
- **Escalate to CMHT options are hard-coded.** The reasons and urgency levels are fixed arrays in `urgent-cases.component.ts`, and the CMHT team is a free-text box. `LookupSeeder` already seeds

`EscalationReason`, `EscalationUrgency` and `CmhtTeam` option lists (with different wording), which admins can edit but which have no effect. Load the dialog from those lists.

- **The MDT Queue menu badge** is loaded once when the shell starts and does not change after items are confirmed, declined or discussed until the page is reloaded. (The Urgent Cases badge now refreshes on both escalation and resolution events.)
- **Built-in role edits are undone on restart** (section 4.6). Decide whether that is wanted; if not, seed built-in roles only when they are created.
- **Outbox delivery is at-most-once after relay.** `OutboxRelayWorker` marks a row processed as soon as it is on the in-memory channel, so an event in flight when the workers stop is lost, and a failed SignalR push is not retried. Rows that fail to relay are retried every 2 seconds with no cap. Processed rows are never purged.
- **`/health` does not check the database.** It proves the API process is up, not that it can reach SQL Server. Add a readiness endpoint with a database check for monitoring.

15.3 Settings that do nothing or disagree

- **Unused settings:** `general.dateFormat` (exposed as `SettingsApiService.dateFormat` but never read), `clinical.followUpDefaultDays` and `clinical.dialogReviewWeeks` (same), and `ui.guestListPageSize` (the guest list uses `PAGE_SIZE = 50` in `guest-data-sheet.component.ts`). Wire them up or remove them from `SettingsCatalog`; the catalogue's own comment says every entry is read somewhere, which is not true.
- **Inactivity threshold:** the setting drives the automatic move to Inactive (its description now says so). The Data Quality "no contact in the last 90 days" check and the `noRecentContact` segment still hard-code 90 days (`ReportReadService`, `GuestReadService.SegmentPredicates`).

15.4 Build warnings

- The production client build succeeds with 24 warnings that Sass `@import` is deprecated (it will be removed in Dart Sass 3.0). They come from `styles.scss` and the dashboard, documents and reports stylesheets. Move them to `@use`.
- 23 component stylesheets exceed the 8 kB warning budget. The largest are `casework-note-drawer.component.scss` (17.8 kB), `dashboard-hub-manager.component.scss` (14.5 kB) and `urgent-cases.component.scss` (13.8 kB). The error budget is 24 kB, so the casework note drawer is the one to split first.

15.5 Testing and delivery

- **Integration tests only check `/health`.** Add API tests against SQL Server in a container (Testcontainers, as the original architecture brief planned), starting with the guest list, the segment predicates, the follow-up queue and the reports.
- **Dapper SQL is untested before production.** The previous developer's machine had no Docker and no SQL Server listening locally, so SQL was first run in production. Make a local or CI SQL Server part of the workflow.
- **No CI.** Add a GitHub Actions workflow that runs `dotnet test`, `npx ng build` and `npx ng test` on every pull request, and protect `main`.
- **No staging environment.** Testing happens on production (document 08).

15.6 Scale and operations

- `ReportMaterializerWorker` recomputes every hub's snapshot from full tables every 5 minutes, with one query per month for contact counts. This is fine at current volumes; move to nightly plus incremental refresh as the architecture brief planned once data grows.
- `AuditEvents` grows with every guest page view and is never archived. SQL Server Express limits a database to 10 GB. Watch the size (document 06) and plan partitioning or a paid edition.
- There is no screen to create or rename hubs; `Hubs` rows are added by SQL (document 06).
- Docker base images (`mcr.microsoft.com/mssql/server:2022-latest`, the .NET 10 images, `node:22-alpine`, `nginx:1.27-alpine`) are only refreshed when rebuilt with `--pull`; `deploy.sh` does not pull. Rebuild with `--pull` monthly for security fixes.
- The repository README is partly out of date: it says Swagger is available in Docker, that password reset only logs the email, and lists fewer screens than exist. Treat this document as the current reference.